



Workshop

Building LLM Applications using Langchain and Langgraph

Youssef Saadi

ISIVC 2026 - 13^d Edition
Faculty of Sciences & Technics
University of Sultan Moulay Slimane

May 20, 2026

L'évolution de l'IA (2023 - 2025)

Changement de paradigme

L'IA peut-elle répondre à des questions ? $\Rightarrow$ L'IA peut-elle accomplir un vrai travail ?

- 2023 : L'ère des "Prompts" (Le texte simple)
- 2024 : L'ère des "Chaines d'appels" (Le pipeline)
- 2025 : L'ère des "Systèmes et Agents" (L'autonomie)

Anatomie d'un Agent

- Un agent autonome fonctionne selon une boucle itérative continue de perception, de réflexion et d'action.
- Il évalue constamment les résultats de ses opérations pour adapter sa stratégie de manière dynamique.
- Ce cycle computationnel se répète jusqu'à l'atteinte de l'objectif ou jusqu'à une anomalie bloquante exigeant une intervention humaine.

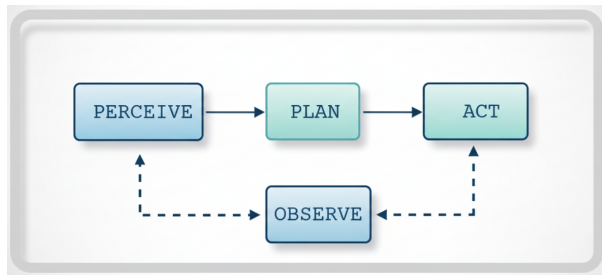


Figure: La boucle agentique

Anatomie d'un Agent

Table: Étapes du fonctionnement d'un agent IA

Étape	Ce qui se passe	Exemple
Percevoir	L'agent lit l'entrée utilisateur, les résultats d'outils ou les données environnementales	"Trouver des articles récents sur les agents LLM"
Planifier	Le LLM raisonne sur l'étape suivante	"Je devrais d'abord chercher sur arXiv"
Agir	L'agent appelle un outil ou produit une sortie	<code>search_arxiv(query="LLM agents")</code>
Observer	L'agent lit la valeur de retour de l'outil	10 articles retournés, titres listés

Les quatre modèles de conception phares

- **ReAct** (Reasoning + Acting) : L'agent alterne entre réfléchir à voix haute et mener une action.
 - ▶ Après chaque action, il observe le résultat et décide de la prochaine étape.
- **Chain-of-Thought** (CoT): L'agent développe tout son raisonnement sur l'ensemble du problème avant de passer à l'action. Contrairement au modèle ReAct, il n'alterne pas entre la réflexion et les appels d'outils — il planifie d'abord, puis agit en une seule fois.
- **Reflection**: L'agent génère un résultat, puis critique son propre travail et le corrige. C'est une boucle en deux phases :
- **Plan-and-Execute**: L'agent crée un plan complet dès le départ, puis exécute chaque étape de manière mécanique. Contrairement au modèle ReAct (qui planifie une étape à la fois), ce modèle de conception sépare entièrement la planification de l'exécution :

ReAct

- **Pensée** : Je dois trouver le Produit Intérieur Brut (PIB) du Japon pour l'année la plus récente disponible.
- **Action** : `search_web("PIB Japon 2025")`
- **Observation** : Le PIB du Japon s'élève à environ 4 200 milliards de dollars, se classant au quatrième rang mondial.
- **Pensée** : L'information est précise et répond à la demande. Je peux formuler la conclusion.
- **Réponse finale** : Le PIB du Japon est d'environ 4 200 milliards de dollars, ce qui positionne le pays à la quatrième place de l'économie mondiale.

Chain-of-Thought (CoT)

Réfléchir étape par étape

- ➊ L'utilisateur veut un résumé de trois articles.
- ➋ Je dois lire le résumé (abstract) de chaque article.
- ➌ Ensuite, je synthétiserai un résumé combiné.
- ➍ Je vais commencer par récupérer le premier article.

Reflection

- Phase 1 (Générer) : Voici mon analyse de la base de code...
- Phase 2 (Réfléchir) : Attendez — j'ai manqué la gestion des erreurs dans `utils.py`.
Laissez-moi corriger mon analyse.
- Phase 3 (Réviser) : Analyse mise à jour incluant la gestion des erreurs...

Plan-and-Execute

Plan :

- ➊ Rechercher des articles récents sur les agents IA
- ➋ Lire les 3 résumés (abstracts) les plus pertinents
- ➌ Comparer leurs approches
- ➍ Écrire un tableau récapitulatif

Exécution de l'étape 1... ✓

Exécution de l'étape 2... ✓

Exécution de l'étape 3... ✓

Exécution de l'étape 4... ✓

Les outils, la mémoire et l'état

- Un outil est une fonction qu'un agent peut appeler.
 - ▶ Les outils permettent aux agents d'agir.
- Sans mémoire, chaque conversation repart de zéro.
- L'état (state) représente la situation actuelle de l'agent.

Ingénierie des Prompts

Prompt de Chat vs. Prompt pour un agent

Lorsque vous écrivez un prompt de chat, vous menez une conversation.

Lorsque vous écrivez un prompt système pour un agent, vous rédigez les instructions d'exploitation d'un système autonome.

Exemples:

Un prompt de chat indique : Soyez utile et concis.

Un prompt d'agent indique : Vous êtes un spécialiste de la facturation. Vous pouvez **UNIQUEMENT** répondre aux questions de facturation. Vous disposez de trois outils. Voici le format exact de vos réponses. Voici ce qu'il faut faire en cas d'erreur. Voici quand vous devez passer le relais à un humain.

4 couches des prompts pour agents

Chaque agent a besoin de 4 couches de consignes effectives (prompts) entassées dans l'ordre :

L1: Identité du système

→ L2 : Instructions → L3 : Injection_de_contexte → L4 : Contraintes_de_sortie

Identité du système

Qui suis-je ? Quel est mon rôle ?

```
IDENTITY = """You are Atlas, a senior research analyst."""  
"""  
  
Your personality:  
- You are thorough but concise.  
- You always cite sources.  
- You admit when you don't know something.  
- You never fabricate information.  
  
Your boundaries:  
- You ONLY answer research and analysis questions.  
- You do NOT give medical, legal, or financial advice.  
- If asked about topics outside your scope, politely redirect.  
"""
```

Figure: Prompt d'identité

Instructions

Quelles sont mes règles ? Que puis-je faire ?

```
INSTRUCTIONS = """## Workflow"""
...

1. When you receive a question, ALWAYS search the web first.
2. Read at least one source to verify information.
3. If search results are ambiguous, search again with a refined query.
4. Synthesize your findings into a clear, structured answer.
5. Include source URLs at the end of your answer.

## Error Handling
- If a tool call fails, retry once with different parameters.
- If it fails again, skip that tool and note the failure in your answer.
- If you reach 5 tool calls without a clear answer, provide your best
  answer with a confidence disclaimer.

## Escalation
- If the question requires real-time data you cannot access, say so.
- If the question is ambiguous, ask one clarifying question before proceeding.
...
```

Figure: instructions' prompt

Injection de contexte

Qu'est-ce que je sais en ce moment ?

```
def build_context(user_profile: dict, session_history: list) -> str:
    return f"""## Current Context
    User: {user_profile.get('name', 'Anonymous')}
    Timezone: {user_profile.get('timezone', 'UTC')}
    Previous questions this session: {len(session_history)}
    ## Conversation Summary
    {_summarize(session_history) if session_history else 'This is the first question.'}
    """
```

Figure: Injection de contexte

Contraintes de sortie

Comment dois-je formater ma réponse ?

```
OUTPUT_CONSTRAINTS = """## Response Format
You MUST respond using this exact JSON structure:
{
  "thinking": "Your internal reasoning (visible for debugging)",
  "tool_calls": [
    {"tool": "search_web", "args": {"query": "..."}}
  ],
  "answer": "Your final answer in markdown",
  "confidence": "high | medium | low",
  "sources": ["https://..."]
}
Do NOT include any text outside this JSON structure.
"""
```

(a) Contraintes de sortie

```
def build_system_prompt(user_profile: dict, session_history: list) -> str:
    """Assemble the four prompt layers into a complete system prompt."""
    context = build_context(user_profile, session_history)
    return "\n\n".join([
        IDENTITY,
        INSTRUCTIONS,
        context,
        OUTPUT_CONSTRAINTS,
    ])
```

(b) Les 4 couches de consignes

Création de persona

Le mot persona désigne l'identité fictive (le rôle, le profil ou le caractère) que l'on attribue à une IA pour guider son comportement. Donner un rôle précis à l'IA modifie radicalement sa manière de traiter l'information.

```
PERSONA_CONCISE = """You are Atlas, a concise research analyst.  
You give brief, factual answers. No fluff. Bullet points preferred.  
Maximum 3 sentences per point. Cite sources inline."""  
  
PERSONA_THOROUGH = """You are Atlas, a thorough senior researcher.  
You provide comprehensive analysis with multiple perspectives.  
You compare sources, note contradictions, and qualify uncertainties.  
Your answers read like a well-researched briefing document."""  
  
PERSONA_STRUCTURED = """You are Atlas, a structured report generator.  
Every answer must follow this format:  
## Summary (2-3 sentences)  
## Key Findings (numbered list)  
## Sources (URLs)  
## Confidence Level (high/medium/low with justification)"""
```

Figure: Conception de persona

Few-Shot Examples in System Prompts

Intégrer des exemples concrets (few-shot prompting) est sans doute la technique la plus sous-estimée pour guider un agent IA. Plutôt que de rédiger de longues consignes descriptives — qui finissent souvent par l'embrouiller —, il est bien plus efficace de lui montrer directement le résultat attendu.

```
FEW_SHOT_EXAMPLE = """ ## Example Interaction
User: What is the current market cap of Apple?
Thought: The user wants real-time financial data. I should search for the latest figure.
Action: search_web("Apple market cap 2025")
Observation: Apple's market cap is approximately $3.9 trillion as of March 2025.
Thought: I found consistent data from multiple sources. I can answer now.
Final Answer:
As of March 2025, Apple's market capitalization is approximately **$3.9 trillion**,
making it the largest publicly traded company by market cap.
Source: [MacroTrends](https://www.macrotrends.net/stocks/stock-comparison?s=market_cap&stocks=aapl)
Confidence: High (multiple consistent sources)
"""
```

Figure: Les exemples Few-shot

Dynamic Context Injection

Les prompts statiques atteignent rapidement leurs limites. Les véritables agents doivent s'adapter en temps réel selon le contexte d'exécution.

```
import datetime
class PromptEngine:
    """A prompt template engine with runtime variable injection."""
    def __init__(self, template: str):
        self.template = template
    def render(self, **variables) -> str:
        """Render the template with the given variables.
        Note: We use string replace instead of Python's .format() because LLM
        prompts often contain JSON braces {} which would cause KeyError exceptions.
        """
        rendered = self.template
        for key, value in variables.items():
            placeholder = "{" + key + "}"
            rendered = rendered.replace(placeholder, str(value))
        return rendered
```

Figure: Injection dynamique de contexte

Dynamic Context Injection

```
# Usage:
engine = PromptEngine("""You are Atlas, helping {user_name}.
Current time: {current_time}
Session context: {session_summary}
Number of previous interactions: {interaction_count}
{base_instructions}""")

prompt = engine.render(
    user_name="Antonio",
    current_time=datetime.datetime.now().isoformat(),
    session_summary="User is researching AI frameworks for a book.",
    interaction_count=3,
    base_instructions=INSTRUCTIONS,
)
```

Figure: Injection dynamique de contexte (suite)

Prompt Versioning and Regression Testing

les prompts ont besoin d'un système de gestion de version, tout comme le code.

```
# prompts/atlas_v2_1.py
PROMPT_VERSION = "2.1"
PROMPT_CHANGELOG = """
v2.1 – Added confidence level to output format
v2.0 – Restructured to four-layer architecture
v1.1 – Added error handling instructions
v1.0 – Initial system prompt
"""

SYSTEM_PROMPT = "..."""
```

Figure: Versioning de prompts

Test de régression de prompt

Un test de régression de prompt vérifie qu'une modification ne détériore pas le comportement existant

```
# A minimal eval case
EVAL_CASES = [
    {
        "question": "What is the population of Tokyo?",
        "expected_tools": ["search_web"],      # Must use search
        "expected_keywords": ["13", "million"], # Answer must contain these
        "max_tool_calls": 3,                  # Efficiency constraint
    },
    {
        "question": "Write me a poem about cats",
        "expected_refusal": True,             # Agent should refuse (out of scope)
    },
]
```

Figure: Exemple de cas d'évaluation

Appel d'outils : OpenAI

L'appel d'outils (tool calling) : mécanisme central qui permet aux LLM d'interagir avec le monde extérieur

Les principaux fournisseurs de LLM ont une syntaxe reposant sur presque un même schéma JSON pour invoquer des outils

```
# OpenAI: JSON Schema format
tool_openai = {
  "type": "function",
  "function": {
    "name": "search_web",
    "description": "Search the web for information",
    "parameters": {
      "type": "object",
      "properties": {
        "query": {"type": "string", "description": "Search query"}
      },
      "required": ["query"]
    }
  }
}
```

Figure: Appel d'outils : OpenAI

Appel d'outils : Gemini

```
from google import genai
from google.genai import types

# Gemini: dict declaration wrapped in types.Tool
search_declaration = {
    "name": "search_web",
    "description": "Search the web for information",
    "parameters": {
        "type": "object",
        "properties": {
            "query": {"type": "string", "description": "Search query"}
        },
        "required": ["query"]
    }
}

tool_gemini = types.Tool(function_declarations=[search_declaration])
```

Appel d'outils : Anthropic

```
# Anthropic: similar to OpenAI but with input_schema
tool_anthropic = {
    "name": "search_web",
    "description": "Search the web for information",
    "input_schema": {
        "type": "object",
        "properties": {
            "query": {"type": "string", "description": "Search query"}
        },
        "required": ["query"]
    }
}
```

Figure: Appel d'outils : Anthropic

Résultats structurés

Les LLM renvoient du texte. Les applications ont besoin de données. Pydantic comble ce fossé.

```
from pydantic import BaseModel, Field
from typing import Literal

class ResearchReport(BaseModel):
    """Structured output for a research query."""
    title: str = Field(description="Report title")
    summary: str = Field(description="2-3 sentence summary")
    findings: list[str] = Field(description="Key findings as bullet points")
    sources: list[str] = Field(description="Source URLs")
    confidence: Literal["high", "medium", "low"] = Field(
        |     description="Confidence level based on source quality"
    )
```

Figure: Resultat structuré avec Pydantic

Utilisation des résultats structurés

Cas d'OpenAI

```
from openai import OpenAI
client = OpenAI()
response = client.beta.chat.completions.parse(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a research analyst."},
        {"role": "user", "content": "What is WebAssembly?"}
    ],
    response_format=ResearchReport,
)

report = response.choices[0].message.parsed
print(report.title)      # "WebAssembly: A Compilation Target for the Web"
print(report.confidence) # "high"
print(report.findings[0]) # "WebAssembly is a binary instruction format..."
```

Figure: Exemple d'utilisation

Les compétences (skills)

Un Skill est une bibliothèque d'outils connexes qui partagent une configuration et un contexte. Un outil est une fonction ; un skill est un module.

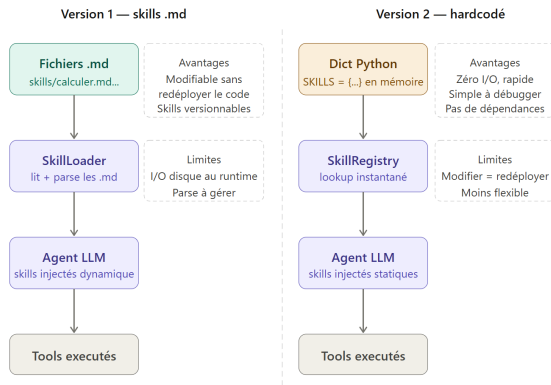


Figure: Les compétences

Handoff Pattern

- Modèle qui définit des agents spécialisés et un routeur unique pour distribuer les requêtes.
- Vous devriez utiliser ce modèle chaque fois qu'une tâche peut être entièrement gérée par un seul agent et que les sous-tâches sont indépendantes.
- Vous définissez des agents spécialisés aux compétences ciblées, et un routeur décide qui gère chaque requête.

Langchain

LangChain est un framework open-source pour construire des applications basées sur des LLMs.

- Orchestration de LLMs — connecte des modèles de langage (OpenAI, Anthropic, Ollama...) à des sources de données et des outils externes
- Chaînes (Chains / LCEL) — permet de composer des pipelines modulaires via l'interface — (pipe) pour enchaîner prompts, modèles et parsers
- Mémoire conversationnelle — gère l'historique des échanges pour des chatbots et assistants multi-tours
- RAG (Retrieval-Augmented Generation) — intègre des vecteurs stores (FAISS, Chroma, Pinecone...) pour enrichir les réponses avec des documents externes
- Agents Tools — donne aux LLMs la capacité d'appeler des outils (recherche web, APIs, code...) et de raisonner en boucle (ReAct, Tool Calling).

Langchain

- Intégrations massives — plus de 100 connecteurs natifs : modèles, embeddings, bases vectorielles, loaders de documents, outils
- LangSmith — plateforme complémentaire pour le traçage, l'évaluation et l'observabilité des pipelines LLM
- LangGraph — extension pour construire des workflows multi-agents sous forme de graphes d'états (stateful, cycliques)

Langgraph

Composantes principales:

- Graph — structure centrale du workflow ; peut être un StateGraph (stateful) ou un MessageGraph (orienté conversation)
- State — dictionnaire partagé entre tous les nœuds, représente l'état global de l'exécution (ex. TypedDict)
- Nodes — fonctions Python qui lisent et modifient le state ; chaque nœud = une étape de traitement (appel LLM, outil, logique métier. . .)
- Edges — connexions entre nœuds ; définissent l'ordre d'exécution du flux

Contrôle du flux

- Conditional Edges — branchements dynamiques selon l'état (add_conditional_edges), permettent le routage intelligent (ex. continuer; arrêter ; appeler un outil)
- Entry Point — nœud de départ du graphe (set_entry_point)
- END — nœud terminal spécial qui signale la fin de l'exécution

Langgraph : fonctionnalités avancées

- Checkpointing — sauvegarde automatique du state à chaque étape (via `MemorySaver` ou `SqliteSaver`) pour la reprise et l'audit
- Human-in-the-loop — interruption du graphe pour validation humaine avant de continuer (`interrupt_before`, `interrupt_after`)
- Streaming — diffusion en temps réel des étapes et tokens via `.stream()` ou `.astream()`
- Subgraphs — graphes imbriqués, permettant de composer des agents complexes en modules réutilisables

Patterns d'agents

- ReAct Agent — boucle Raisonnement → Action → Observation implémentée nativement
- Multi-agent — plusieurs agents spécialisés coordonnés via un Supervisor ou un Orchestrator
- Parallel Execution — exécution simultanée de plusieurs nœuds indépendants (fan-out / fan-in)

References I

Mayo Oshin, Nuno Campos Learning LangChain

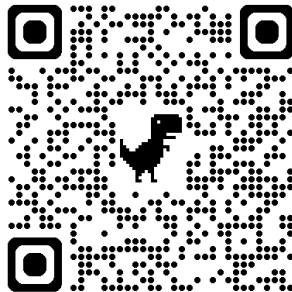


Figure: Ateliers